

Exercício para aquecimento

1. Quantos processos são criados quando o programa abaixo é executado?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    /* fork a child process */
    fork();
    /* fork another child process */
    fork();
    /* and fork another */
    fork();
    return 0;
}
```

Exercício para aquecimento

1. Quantos processos são criados quando o programa abaixo é executado?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    /* fork a child process */
    fork();
    /* fork another child process */
    fork();
    /* and fork another */
    fork();
    return 0;
}
```

```
1 (processo inicial)
├─ 2 [fork#1, filho de 1]
│   └─ 3 [fork#2, filho de 2]
│       └─ 4 [fork#3, filho de 3]
│           └─ 5 [fork#3, filho de 2]
├─ 6 [fork#2, filho de 1]
│   └─ 7 [fork#3, filho de 6]
└─ 8 [fork#3, filho de 1]
```

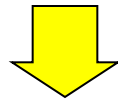
INF01151 – Sistemas Operacionais II

Aula 09 - Comunicação inter-processos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Programação concorrente surgiu com a multiprogramação
 - Mecanismos básicos supõem memória compartilhada, mas isso nem sempre é possível (ou desejável)
 - Questões de segurança/privacidade
 - Máquinas com memória distribuída (e.g. redes de computadores)
- Processos UNIX têm espaço de endereçamento próprio e privativo
 - Não há compartilhamento de memória, por exemplo, `fork()` cria processo filho com espaços de endereçamento distinto (variáveis são copiadas)



Mecanismos de IPC (*Inter-Process Communication*)

Nota: as threads criadas por um mesmo processo compartilham o espaço de endereçamento daquele processo

Processos interativos

- **Tipos de processos**

- **Processos independentes**: não podem afetar ou serem afetados pela execução de outro processo (não compartilham dados)
- **Processos interativos**: podem afetar ou serem afetados pela execução de outro processos (compartilham dados)

Benefícios da interação entre processos

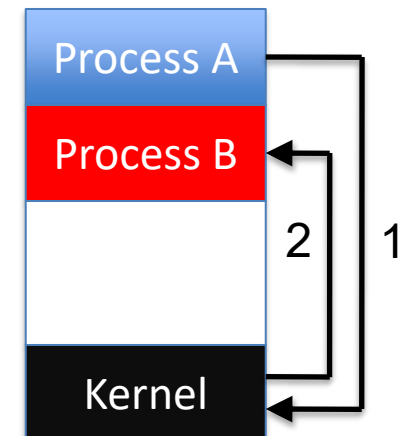
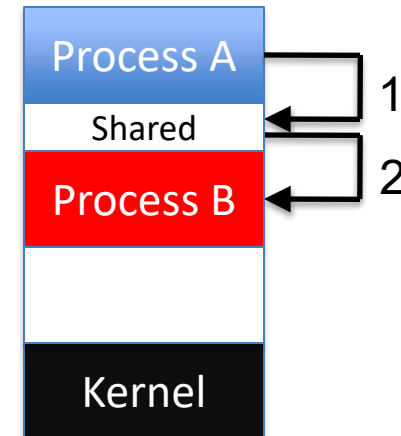
- Compartilhamento de informações
- Speed-up da computação
- Modularidade
- Conveniência

Perigos da interação entre processos

- Dados corrompidos, deadlocks, aumento da complexidade
- Necessidade que processos sincronizem seu processamento

Comunicação inter-processos

- Processos competitivos/colaborativos precisam de mecanismos IPC
 - Troca de dados e compartilhamento de informações
- Dois modelos fundamentais para IPC
 - **Memória compartilhada** (shared-memory systems)
 - Região de memória compartilhada entre processos é estabelecida
 - Processos compartilham informações através da leitura e escrita de dados na região
 - Read/Write
 - **Troca de mensagens** (message-passing systems)
 - Comunicação ocorre através de mensagens enviadas de um processo para outro
 - Send/Receive



Comunicação inter-processos

- Troca de mensagens vs. memória compartilhada
- Ambos são bastante comuns em SOs atuais, em geral...
 - Troca de mensagens
 - Requer chamadas de sistema, **intervenção do kernel**
 - Melhor para compartilhamento de **pequenas quantidades** de dados
 - Solução utilizada em **sistemas distribuídos**
 - Memória compartilhada
 - Chamadas de sistema necessárias apenas para estabelecer região
 - Após, qualquer acesso é realizado **sem assistência do kernel**
 - Comunicação mais **rápida e conveniente**

Principais mecanismos de IPC

- Comunicação inter-processos
 - Simplesmente informar outro processo que algum evento ocorreu; ou
 - Envolver transferência de dados de um processo para outro
 - Mecanismos de IPC
 - Pipes
 - Sinais
 - Filas de mensagens
 - Shared-memory
 - Semáforos (**estudados na aula 05**)
 - Sockets (**serão vistos na aula 11**)
 - RPC/RMI (**serão vistos na aula 14**)
 - Web sockets (**serão vistos na aula 15**)
 - Web services (**serão vistos na aula 16**)
- Todos permitem que processos troquem informações.
- Questão: como escolher?**
- Alguns são mais adequados para *aplicações particulares*
- Processos relacionados?
 - Comunicam por rede?
 - Unidirecional/bidirecional?
 - Mensagens curtas?
 - Acesso rápido?
 - Linguagem de programação?
 - Portabilidade?
 - ...

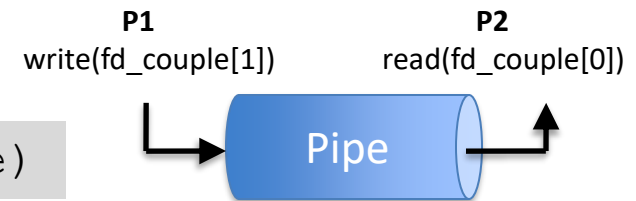
Pipes

```
#include <unistd.h>
```

- Provê um fluxo de dados **unidirecional** entre dois processos
 - Um processo escreve no pipe, outro processo lê do pipe
 - Implementado com um par de descritores de arquivo
 - O que é escrito no primeiro descritor pode ser lido no segundo (ordem FIFO)

- Criado por chamada de sistema

```
int pipe(int *fd_couple)
```



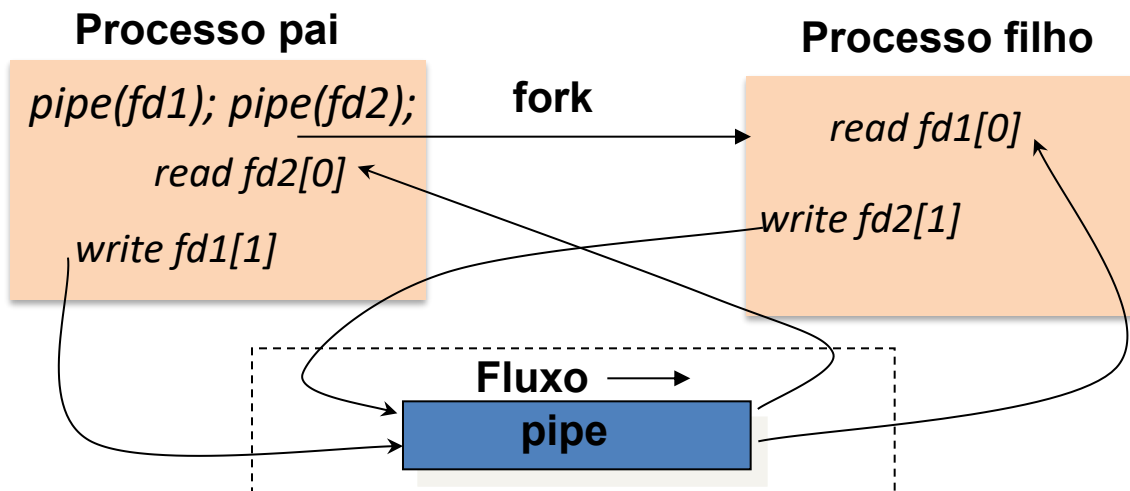
- Cria pipe e armazena descritores de arquivos `fd_couple[0]` (R) e `fd_couple[1]` (W)
 - Retorna -1 se ocorrer erro, 0 caso contrário
- Pipe é acessado por um dos descritores de arquivos
- Lido e escrito através de chamadas **read** e **write** convencionais
- Se comunicação **bidirecional** for necessária?
 - É preciso **usar 2 pipes**, cada pipe usado para enviar dados em uma direção

Detalhes sobre pipes simples

- Devem ser compartilhados pelos processos que desejam se comunicar (**apenas processos relacionados**)
- Situação genérica:
 - Processo cria dois pipes, e faz um `fork()` para criar processo filho
 - Filho herda descritores de arquivos abertos
 - Pai e filho utilizam os pipes

Alternativa: Named Pipes (FIFO)

- Comunicação pode ser bidirecional
- Relação pai/filho não é necessária

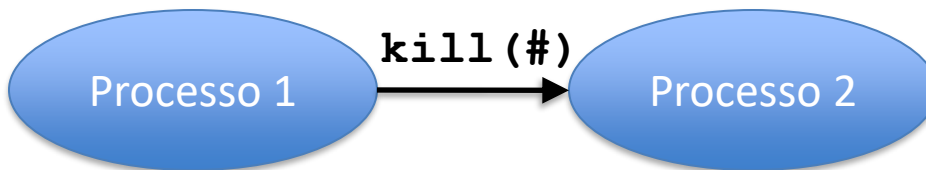


IMPORTANTE

- ① Cada pipe é uni-direcional apenas
- ② Atomicidade garantida para msg menor que PIPE_BUF bytes
- ③ Pipes meio abertos chamados de *widowed* (ou *broken*)
 - Escrita causa erro
 - Leitura retorna 0 bytes

Sinais (signals)

- Um dos primeiros mecanismos de IPC disponíveis no UNIX
 - Tipo de mensagem bastante **primitivo**
- Sinais podem interromper o fluxo de controle normal de um processo, mas **não tem capacidade de envio de dados**
 - Servem para indicar a ocorrência de um evento
 - Única informação gerada é o **número/tipo do sinal**
- Originalmente, sinais eram de um só tipo (como um *beep*) Como?
 - Quando recebido por processo, processo sempre reagia do mesmo modo
 - Historicamente, reação era terminar o processo → função para envio de sinais: `kill()`
 - Sinais foram tipificados, permitindo reagir de diferentes maneiras



Tratamento de sinais

<i>Tipo</i>	<i>Handler</i>
1	func_x()
2	func_y()

Geração de sinais

- Tipos de sinais suportados dependem da arquitetura do processador
 - Comportamento definido no padrão POSIX
- Tipos de sinais enviados a um processo
 - **Exceções**: núcleo notifica o processo falha de segmentação (SIGSEGV)
 - **Outros processos**: um processo encerra outro (SIGKILL)
 - **Notificações**: ocorrência de eventos como dispositivo pronto
 - **Alarmes**: programação de geração de alarmes (SIGALRM)
- UNIX e derivados de SVR4 e 4.4 BSD possuem 31 sinais
- API UNIX

```
#include <signal.h>
```

 - **signal()**: associa uma função de tratamento a tipo de sinal
 - **kill()**: envia um sinal

Tratamento de sinais

- Processo pode tratar o sinal de **três modos**
 - ① **Ignorar o sinal**
 - Exceto SIGSTOP e SIGKILL
 - ② **Capturar o sinal**
 - Implica na invocação de uma rotina específica para tratamento do sinal
 - ③ **Executar ação *default* definida pelo núcleo para aquele sinal**
 - Por default, núcleo define uma de 5 ações a ser executada
- **Ações *default***
 - **Abortar:** termina o processo imediatamente
 - **Memory dump:** termina o processo, mas antes gera *core dump*
 - **Ignorar:** simplesmente ignorar o sinal
 - **Suspender o processo:** também usada para depuração
 - **Continuar:** processo que estava previamente suspenso
 - Reverte ação prévia, passando estado de suspenso para pronto

Atendimento de sinais

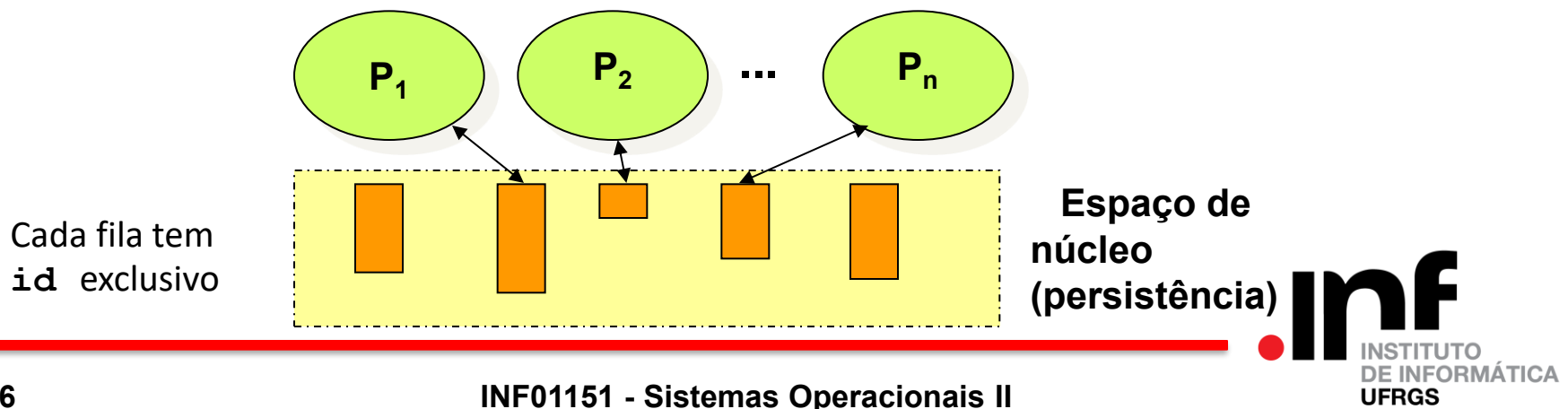
- Quem pode o que?
 - Processo A pode enviar sinal a processo B somente **se pertencem ao mesmo usuário**
 - Processos executando como superuser podem sinalizar **qualquer processo**
 - Sinais enviados **pelo próprio SO** podem não estar sujeitos a **tratamento customizado**
 - SIGSTOP e SIGKILL sempre terminaram o processo
- Quando gerados, os sinais ficam no estado “pendente” até serem encaminhados para o processo destino
 - Normalmente, o encaminhamento é rápido...

Atendimento de sinais - Importante

- Para notificar sinais, o sistema operacional tem uma **máscara de bits** para cada processo
 - Quando processo receber o sinal n , n -ésimo bit na máscara é marcado
 - Kernel chama função **issig()** para verificar sinais pendentes
 - Antes de processo bloquear e imediatamente após acordar
 - Antes de voltar para modo usuário (chamada de sistema ou interrupção)
 - Quando a função de tratamento terminar, o bit é repostado a 0
- Porém...
 - Se **issig()** não for executada antes que outros sinais do mesmo tipo cheguem, apenas o último será percebido pelo processo
 - Os outros sinais daquele tipo serão perdidos

Filas de mensagens

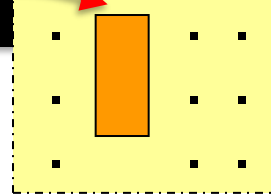
- Mecanismo que permite processos trocarem informações entre si
 - De forma completamente **assíncrona**
 - **Caixa postal** onde processos podem **depositar** e **retirar** mensagens
- Mensagens são compostas por
 - **Tipo de mensagem**
 - Não são definidas pelo núcleo; processos especificam seus próprios tipos de mensagens
 - **Área de dados de tamanho variável**
- Organizadas em filas de mensagens até que processo possa receber
 - Podem ser compartilhadas por processos que não são pai e filho



Filas de mensagens

- Descritores de **fila de mensagens**

- Fila de mensagens pendentes
- Fila de processos a espera de mensagens (receptores)
- Fila de processos a espera para enviar (emissores)
- Dados sobre tamanho e conteúdo da fila de mensagens



- Procedimento de envio e entrega

- Quando processo adiciona mensagem na fila, núcleo verifica lista de receptores esperando mensagens daquele tipo
 - Uma mensagem é entregue para **todos** processos **receptores** que a esperam
 - Se não houver processo esperando por mensagem daquele tipo
 - >> **Havendo espaço**: mensagem fica na **fila de mensagens** pendentes, e mensagens ficam armazenadas na fila até serem lidas
 - >> **Não havendo espaço**: processo se auto-insere na fila de **processos emissores** a espera para enviar
 - Persistência no kernel
- Qualquer processo pode* ler e escrever em qualquer fila
 - *Depende de autorização e permissão

API System V $\neq$ API POSIX

`send (msg, priority)`

Prioridade é usada para entrega de mensagens

SYSTEM V: usuário requisita explicitamente msg com prioridade específica (prioridade é usada como “endereço” da caixa postal)

POSIX: mensagem mais antiga com prioridade mais alta é entregue

Shared-memory

- Permite processos compartilhar uma área comum de memória
 - Comunicam-se através de leitura e escrita na área compartilhada
 - Processos devem garantir que não irão escrever no mesmo local simultaneamente
 - Como???
- Características
 - Mecanismo de simples compreensão para compartilhar dados
 - Não há transferência de dados
 - Acesso se faz através de operações de atribuição e ponteiros
 - Uma vez estabelecida a região compartilhada, acesso é feito em espaço de usuário sem intervenção do núcleo
 - Melhor desempenho por evitar trocas de modo/contexto

Shared-memory

- UNIX suporta duas interfaces padronizadas
 - **Shared-memory System V**
 - Região reside no espaço de endereçamento do processo criando o **segmento de memória compartilhada**
 - Outros processos que desejem se comunicar usando esse segmento **precisam se “anexar”** (*attach*) ao espaço de endereçamento
 - **Shared-memory POSIX**
 - Região é armazenada como um **arquivo temporário** montado em `/dev/shm`
 - Equivale a acessar arquivos (`open`, `read/write`, `close`)
 - Duas primitivas
 - `shm_open ( )`: cria um ponteiro para uma região de memória compartilhada
 - `shm_unlink ( )`: destrói ponteiro para região compartilhada

Shared-memory (System V)

- Criação de segmento de memória compartilhada

- Chamada shmget()

// SHared Memory GET

```
segment_id = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);
```

Inteiro que
identifica o
segmento de
memória

- 1º parâmetro: key para o segmento; se for IPC_PRIVATE, novo segmento é criado
- 2º parâmetro: tamanho em bytes do segmento
- 3º parâmetro: modo de uso, i.e., read, write ou ambos

- Para que processo consiga acessar segmento, deve anexá-lo

- Chamada shmat()

// SHared Memory ATtach

```
shared_memory = (char *) shmat(segment_id, NULL, 0);
```

Ponteiro p/
início da
memória onde
a região
compartilhada
foi anexada

- 1º parâmetro: inteiro que identifica segmento de memória
- 2º parâmetro: ponteiro indicando onde a memória compartilhada será anexada
 - NULL = escolha do SO
- 3º parâmetro: flag indicando se região será read-only ou write-only
 - 0 = ambos

Shared-memory (System V)

- Após memória compartilhada ter sido anexada ao espaço de endereçamento de um processo
 - Acesso é feito como um acesso de leitura/escrita convencional
- Segmento compartilhado é identificado por `segment_id`
 - Deve ser conhecido pelos processos que o compartilham
 - Mantido pelo núcleo até que o segmento seja destruído e que não existam processos usando
- Primitivas adicionais:
 - `shmdt()` : desanexa segmento compartilhada
 - `shmctl()` : remove segmento de memória compartilhada do sistema

```
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main (int argc, char *argv[]) {

    in segment_id;
    char *shared_memory;
    const int size = 4096;

    /* allocate shared memory segment */
    segment_id = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);

    /* attach the shared memory segment */
    shared_memory = (char *) shmat(segment_id, NULL, 0);

    /* write a message to the shared memory segment */
    sprintf(shared_memory, "Hi there!");

    /* now print out the string from shared memory */
    printf("%s\n", shared_memory);

    /* detach the shared memory segment */
    shmdt(shared_memory);

    /* remove the shared memory segment */
    shmctl(segment_id, IPC_RMID, NULL);

    return 0;
}
```

Leituras adicionais

- Deitel, Deitel e Choffnes, Sistemas Operacionais, 3a edição, Pearson Prentice Hall
 - Capítulo 20 (seção 20.10)
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 3 (seções 3.4 e 3.5)
- Interprocess communication / UNIX IPC programming
 - <http://mij.oltrelinux.com/devel/unixprg/>